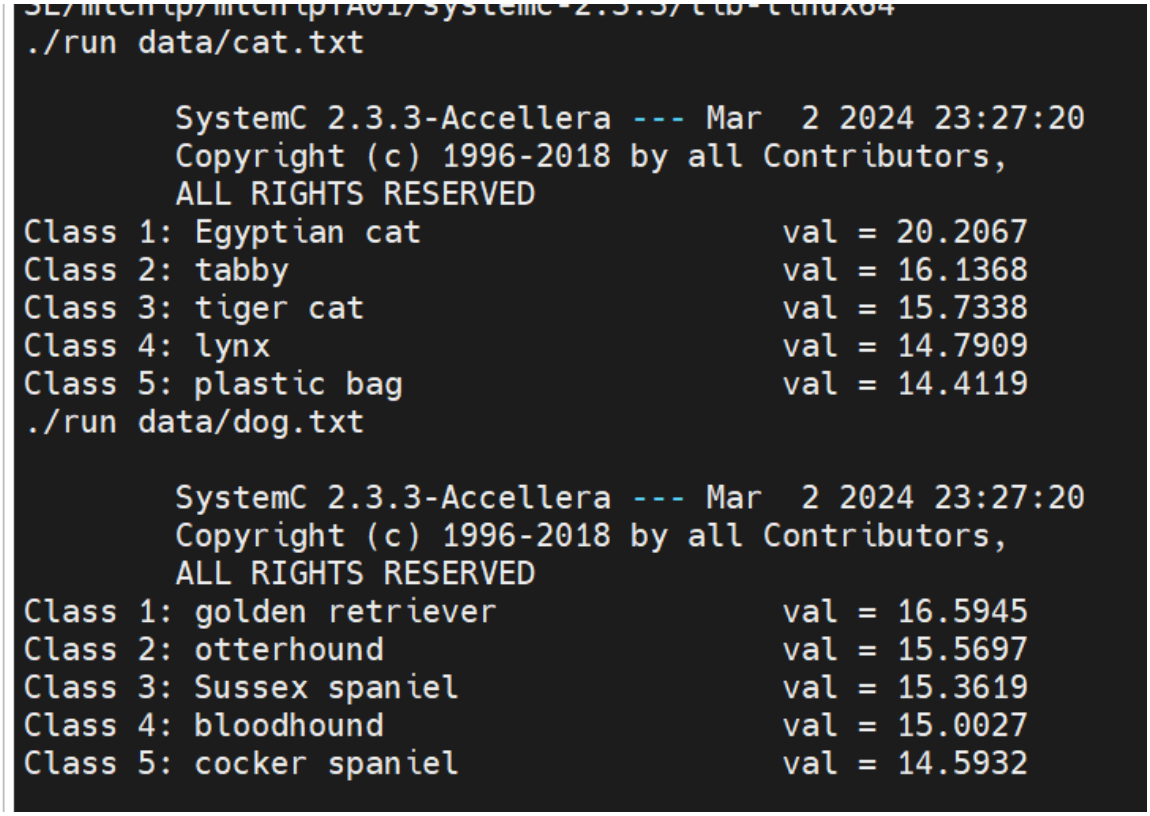
ML chip Hw1

Simulation result



Implementation

The Alexnet is implemented in forms of sc_modules. Modules for functional blocks such as Conv2d, max pool, and ReLU are created, and are cascaded in the sc_main fuction to form the whole network. A monitor module is also deployed to print out the output message.

```
    // sc_module instances that make up the Alexnet
    Conv2d conv1("conv1", 224, 55, 3, 64, 2, 4, 11, "conv1");
    ReLU_3D relu1("relu1", 64, 55);
    MaxPool2d pool1("pool1", 64, 55, 3, 2);
    Conv2d conv2("conv2", 27, 27, 64, 192, 2, 1, 5, "conv2");
```

```
    ReLU_3D relu2("relu2", 192, 27);
    MaxPool2d pool2("pool2", 192, 27, 3, 2);
    Conv2d conv3("conv3", 13, 13, 192, 384, 1, 1, 3, "conv3");
    ReLU_3D relu3("relu3", 384, 13);
    Conv2d conv4("conv4", 13, 13, 384, 256, 1, 1, 3, "conv4");
    ReLU_3D relu4("relu4", 256, 13);
    Conv2d conv5("conv5", 13, 13, 256, 256, 1, 1, 3, "conv5");
    ReLU_3D relu5("relu5", 256, 13);
    MaxPool2d pool5("pool5", 256, 13, 3, 2);
    Flatten flatten("flatten");
    Linear fc1("fc6", 9216, 4096, "fc6");
    ReLU_1D relu6("relu6", 4096);
    Linear fc2("fc7", 4096, 4096, "fc7");
    ReLU_1D relu7("relu7", 4096);
    Linear fc3("fc8", 4096, 1000, "fc8");

    // The module in charge of printing the ouput msg
    Monitor monitor("monitor");
```

Observations

The size of Conv2d, ReLU, and Linear can be dynamically adjusted to adapt to diffrent input and output size. As a result, an extra destructor and an parameterized constructor are needed. Take the class Conv2d for example:

```
SC_MODULE(Conv2d)
{
    const int in_wid, out_wid, in_ch, out_ch;
    const int padding, stride, kernel_size;
    string file_name;
    sc_in<double> ***in_data;
    sc_out<double> ***out_data;

    // weight matrix
    vector<vector<vector<vector<double>>>> weight;
```

```
// bias vector
vector<double> bias;

void run()
{
            // skipped
}

SC_CTOR(Conv2d) : in_wid(0), out_wid(0), in_ch(0), out_ch(0)
{
    //  not used
    in_data = NULL;
    out_data = NULL;
}

Conv2d(sc_module_name name, int in_wid, int out_wid, int in_
{
        // dynamically allocate memory for in_data and out_d
    in_data = new sc_in<double> **[in_ch];
    for (int i = 0; i < in_ch; i++)
    {
        in_data[i] = new sc_in<double> *[in_wid];
        for (int j = 0; j < in_wid; j++)
        {
            in_data[i][j] = new sc_in<double>[in_wid];
        }
    }

    out_data = new sc_out<double> **[out_ch];
    for (int i = 0; i < out_ch; i++)
    {
        out_data[i] = new sc_out<double> *[out_wid];
        for (int j = 0; j < out_wid; j++)
        {
            out_data[i][j] = new sc_out<double>[out_wid];
```

```
            }
        }

             /*-------skipped-----------*/
    }

    ~Conv2d()
    {
        // dellocate all the arrays
        for (int i = 0; i < in_ch; i++)
        {
            for (int j = 0; j < in_wid; j++)
            {
                delete[] in_data[i][j];
            }
            delete[] in_data[i];
        }
        delete[] in_data;

        for (int i = 0; i < out_ch; i++)
        {
            for (int j = 0; j < out_wid; j++)
            {
                delete[] out_data[i][j];
            }
            delete[] out_data[i];
        }
        delete[] out_data;
    }
};
```

Convenient as this measure is, there is also downside to it since all the Conv2d instances have to be constructed before simulation and deleted afterward, which takes about 10 minutes to complete. This takes more time than the simulation itself, and I believe there should be a better approach for this, like declaring flattened 1d arrays instead of 3d arrays.

Challenge encountered

In the context of this lab, the biggest problem I had faced is obviously that I was unfamiliar with how the systemc package works. For example, sc_signals cannot be declared as vectors whose size have been pre-defined since it will illegally calls the ctor for sc_signal, which is private. Further more, I was too concentrated on getting the functionality right that I forgot to take care of the timing, and since all of my modules are connected as combinational circuits, the monitor module was triggered multiple times during the simulation. But because all the combinational operation were finished within 0s, I could fix this by simply using SC_THREAD instead of SC_METHOD in the monitor module and have the thread function wait for 1 ns so that the monitor will only output once.

```
SC_MODULE(Monitor)
{
    sc_in<double> *prob;

    void run()
    {
        wait(1, SC_NS);
        // cout << "current time: " << sc_time_stamp() << endl;
        // cout << "run monitor" << endl;
        ifstream in_file("data/imagenet_classes.txt");
        vector<string> classes;
        // parse the class names
        while (!in_file.eof())
        {
            string class_name;
            getline(in_file, class_name);
            classes.push_back(class_name);
        }
        in_file.close();

        // pick the top 5 classes with the highest probabilities
        vector<double> prob_vec;
        for (int i = 0; i < 1000; i++)
```

```
        {
            prob_vec.push_back(prob[i].read());
        }

        vector<double> prob_vec_sorted = prob_vec;
        sort(prob_vec_sorted.begin(), prob_vec_sorted.end(), gre

        for (int i = 0; i < 5; i++)
        {
            int index = find(prob_vec.begin(), prob_vec.end(), p
            cout << "Class " << i + 1 << ": " << left << setw(30
            cout << " val = " << left << setw(10) << prob_vec_so
        }
    }

    SC_CTOR(Monitor)
    {
        // cout << "constructor for Monitor" << endl;
        prob = new sc_in<double>[1000];
        // SC_METHOD(run);
        SC_THREAD(run);
        for (int i = 0; i < 1000; i++)
        {
            sensitive << prob[i];
        }
    }

    ~Monitor()
    {
        delete[] prob;
    }
};
```